Open Access | https://doi.org/10.25080/wuxr5665

Docling for Multimodal Retrieval

Mahima Arora[1], and **Aarti Jha**[1]

[1]Red Hat India Pvt. Ltd.

SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Abstract

Retrieval-Augmented Generation (RAG) has become a widely adopted approach for grounding large language models in external knowledge sources. However, most RAG pipelines remain text-centric, overlooking diagrams, figures, tables, and layout structures that contain critical information in scientific, technical, and enterprise documents. This paper presents a multimodal document parsing framework built on Docling, a layout-aware and format-agnostic document understanding library, that preserves document structure by extracting text, tables, and images as distinct retrieval units. The framework employs modality-specific representations: hierarchy-aware text chunks, image chunks enriched with vision-language descriptions, and table chunks that retain relational structure while supporting deterministic SQL querying. These representations are indexed in a unified vector database and served to a tool-augmented agent capable of retrieving and reasoning over textual, tabular, and visual information. Evaluation on scientific and technical documents shows improvements over a text-only baseline in context recall, faithfulness, and answer correctness, with the largest gains on image and table queries.

Keywords retrieval-augmented generation, multimodal document understanding, document parsing, vector databases, large language models, information retrieval

1. Introduction

Retrieval-Augmented Generation (RAG) is a technique that enhances large language models (LLMs) by retrieving relevant information from external knowledge sources at inference time, grounding model responses in factual, domain-specific content without requiring costly retraining. It has become a foundational approach for building accurate, knowledge-intensive applications, and its standard pipeline follows a well-established pattern: documents are loaded, split into text chunks, embedded into a vector database, and retrieved to condition generation. While effective for purely textual corpora, this paradigm breaks down when applied to the complex, multimodal documents that dominate real-world knowledge bases.

Scientific papers, technical manuals, and enterprise reports communicate knowledge through far more than prose. Architecture diagrams, performance tables, charts, and structured layouts often carry the most critical information in such documents. Yet conventional RAG pipelines silently discard these elements during ingestion, reducing rich documents to fragmented text. This introduces two concrete failure modes. The first is a *context gap*: when narrative text references a figure or diagram that was dropped during parsing, the retrieved context is semantically incomplete and the model cannot reason over evidence it never received. The second is the *table trap*: flattening structured tables into linear token sequences destroys row-column relationships and renders the resulting text chunks neither accurately embeddable nor reliably retrievable.

Published Jul 13, 2026

Correspondence to
Mahima Arora
maharora@redhat.com

Open Access

Copyright © 2026 Arora & Jha. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

Addressing these limitations requires treating documents not as streams of text, but as coordinated ecosystems of distinct content types: narrative text, visual elements, and relational data. Each modality demands its own representation and retrieval strategy.

This paper presents a multimodal document parsing and retrieval framework built on Docling, a layout-aware document understanding library. The framework introduces a typed chunking architecture with modality-specific enrichment. Text chunks use a hierarchy-aware hybrid chunking strategy that preserves section context. Image chunks are enriched with vision-language descriptions and classified by semantic type to support targeted retrieval. Table chunks preserve full relational structure as pandas DataFrames, paired with generated descriptions for semantic retrieval and a SQL execution interface for deterministic querying. These typed chunks are indexed in a unified vector database and served to a tool-augmented agent capable of synthesizing answers across all three modalities.

The key contributions of this work are:

1. A formal characterization of the structural failure modes of text-only RAG on complex documents
2. A layout-aware multimodal parsing pipeline built on Docling that extracts text, images, and tables as first-class retrieval units
3. Modality-specific enrichment strategies including vision-language captioning and SQL-backed table querying
4. A tool-augmented agent architecture that combines semantic retrieval with deterministic execution to enable faithful reasoning across modalities
5. An open-source implementation facilitating reproducibility and community adoption

Together, these contributions advance multimodal document understanding as an essential capability for RAG systems operating over real-world scientific, technical, and enterprise document collections.

2. Related Work

R. Zhao *et al.* [1] survey retrieval-augmented generation across modalities including images, code, tables, graphs, and audio, establishing a taxonomy of algorithmic approaches. At the framework level, several systems address multimodal content in RAG pipelines using patterns related to those in this work. LangChain's multi-vector retriever [2] embeds generated text summaries of tables and images for semantic search while storing original content in a separate document store for downstream synthesis. This caption-and-embed pattern makes non-text elements discoverable but does not prescribe modality-specific access patterns at query time; retrieved tables are passed as raw content to the LLM rather than queried deterministically. LlamaIndex [3] provides query engines that translate natural language into executable code against structured data, including `NLSQLTableQueryEngine` for SQL-based access and `SQLAutoVectorQueryEngine` for combining vector retrieval with SQL execution. These enable deterministic tabular querying but focus on text and structured data without incorporating visual content.

This framework builds directly on Docling [4], which provides the layout-aware parsing backbone, including its `HybridChunker` for hierarchy-aware text segmentation and vision-language captioning for image description. What Docling does not provide is the retrieval and reasoning layer: a typed chunk schema unifying text, images, and tables in a single vector space; SQL-backed table querying for deterministic access; and a tool-augmented agent that selects the appropriate access pattern per modality at query time. Like the multi-vector retriever, the framework uses generated descriptions for cross-modal semantic search; like LlamaIndex's query engines, it supports deterministic SQL execution. The con-

tribution is this end-to-end architecture integrating parsing, modality-specific enrichment, unified retrieval, and agent-based reasoning.

3. Methodology

3.1. Document Parsing with Docling

The framework uses Docling [4], a layout-aware document understanding library that performs full layout analysis across PDF, DOCX, PPTX, and XLSX formats. Docling's pipeline extracts three content modalities: (1) **text** with preserved document hierarchy and section context, (2) **tables** using TableFormer structure recognition to recover row-column relationships, and (3) **images** with vision-language generated descriptions that make visual content semantically searchable. All formats are normalized to PDF for consistent processing, and the parser outputs structured document objects containing text blocks, table objects, and image elements, each annotated with page number, section context, and content type.

3.2. System Architecture

The framework transforms Docling's structured document representation into a multimodal retrieval system through five stages: parsing, chunking, indexing, retrieval, and agent-based answer generation. Figure 1 shows the complete pipeline.

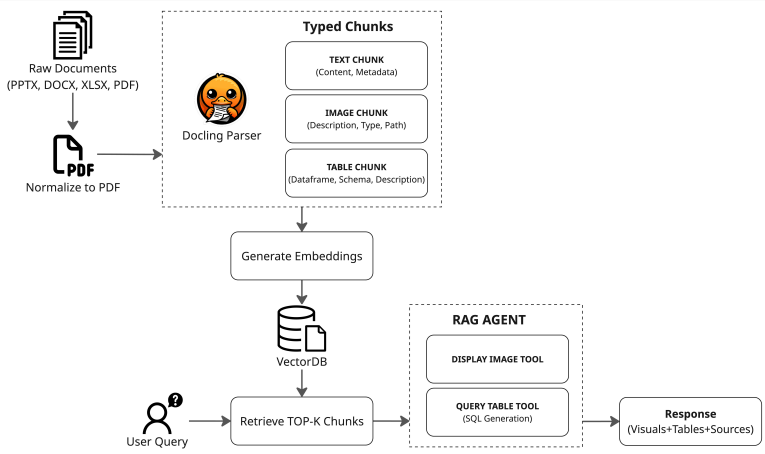


Figure 1. *System architecture: document ingestion, Docling parsing, modality-specific chunking, vector indexing, and agent-based answer generation.*

Three design principles guide this architecture: (1) modality preservation (maintain native representations rather than forcing everything into text), (2) retrieval optimization (generate descriptions that emphasize what questions each element can answer), and (3) deterministic access (provide SQL and display tools for precise operations that embeddings cannot reliably support).

3.3. Chunking Strategy

The chunking stage addresses a fundamental question: how should multimodal document elements be represented for semantic retrieval? Text-only systems flatten everything into character sequences, destroying the structural properties that make tables queryable and visual elements interpretable. The proposed approach defines a typed chunk architecture where each modality receives a representation suited to its information characteristics.

All chunks share a common base schema that enables unified retrieval while preserving modality-specific structure. The key design constraint is that text, images, and tables must be searchable in the same vector space. Rather than joint pixel-text or table-text embeddings, this is achieved by applying standard text embeddings to a `content` field that stores a natural language representation for every modality: narrative prose for text chunks, vision-language generated descriptions for images, and schema-aware descriptions for tables. The base schema tracks provenance and provides this unified embedding interface:

```
class BaseChunk(BaseModel):
    chunk_id: str
    chunk_type: ChunkType  # TEXT, TABLE, or IMAGE
    content: str  #  (text or image/table description)
    sequence_number: int
    source_document: Optional[str]
    source_page: Optional[int]
    parent_heading: Optional[str]  # preserves document hierarchy
```

This unified interface allows all chunk types to be indexed together via the `content` field, while specialized subclasses add modality-specific fields (DataFrames for tables, image paths for visuals) that enable deterministic operations beyond semantic search.

3.3.1. *Text Chunks*

```
# Sample text chunk
if text_chunks:
    c = text_chunks[10]
    print("\n[TEXT CHUNK]")
    print(c)
```

```
[TEXT CHUNK]
chunk_id='text_10' chunk_type=<ChunkType.TEXT: 'text'> content="3.1 Open-domain Question Answering\nOpen-domain question answering (QA) is a
n important real-world application and common testbed for knowledge-intensive tasks [20]. We treat questions and answers as input-output tex
t pairs ( x, y ) and train RAG by directly minimizing the negative log-likelihood of answers. We compare RAG to the popular extractive QA pa
radigm [5, 7, 31, 26], where answers are extracted spans from retrieved documents, relying primarily on non-parametric knowledge. We also co
mpare to 'Closed-Book QA' approaches [52], which, like RAG, generate answers, but which do not exploit retrieval, instead relying purely on
parametric knowledge. We consider four popular open-domain QA datasets: Natural Questions (NQ) [29], TriviaQA (TQA) [24]. WebQuestions (WQ)
[3] and CuratedTrec (CT) [2]. As CT and WQ are small, we follow DPR [26] by initializing CT and WQ models with our NQ RAG model. We use the
same train/dev/test splits as prior work [31, 26] and report Exact Match (EM) scores. For TQA, to compare with T5 [52], we also evaluate on
the TQA Wiki test set." sequence_number=10 source_document='rag_paper.pdf' source_page=4 parent_heading='3.1 Open-domain Question Answering'
extraction_timestamp=datetime.datetime(2026, 2, 3, 18, 50, 59, 676946) word_count=170 char_count=1077
```

Figure 2. *Text chunk schema with hierarchy-aware structure and parent heading context.*

Text chunks solve the problem of preserving narrative context across document structure. Fixed-size chunking (e.g., every 512 tokens) fragments coherent sections and concatenates unrelated paragraphs, breaking the semantic units humans rely on for understanding. The text chunking strategy preserves hierarchy by tracking the parent heading and uses intelligent merging to respect document structure:

```
chunker = HybridChunker(
    max_tokens=2048,
    merge_peers=True,
    merge_threshold=0.5,
)
```

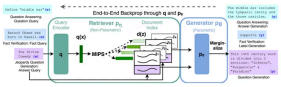
The chunker merges text blocks within the same section when semantic similarity exceeds the threshold, creating chunks that align with how documents are actually organized. A methods section stays together; introduction and conclusion remain separate. The `content` field contains the merged text, and `parent_heading` preserves the section title, allowing retrieval to return contextually grounded chunks rather than orphaned fragments.

3.3.2. *Image Chunks*

```
# Sample image chunk
from IPython.display import Image as IPImage, display

if image_chunks:
    c = image_chunks[0]
    print("\n[IMAGE CHUNK]")
    if c.image_path:
        display(IPImage(filename=c.image_path, width=400))
    print(c)
```

```
[IMAGE CHUNK]
```



```
chunk_id='image_34' chunk_type=<ChunkType.IMAGE: 'image'> content='This is a **system architecture diagram** illustrating a retrieval-augmen
ted generation (RAG) model.\n\nThe main subject is the **end-to-end backpropagation through a retriever and a generator**\n\nCaption: Figure
1: Overview of our approach. We combine a pre-trained retriever ( Query Encoder + Document Index ) with a pre-trained seq2seq model ( Genera
tor ) and fine-tune end-to-end. For query x , we use Maximum Inner Product Search (MIPS) to find the top-K documents z i . For final predict
ion y , we treat z as a latent variable and marginalize over seq2seq predictions given different documents.' sequence_number=34 source_docum
ent='rag_paper.pdf' source_page=2 parent_heading='1 Introduction' extraction_timestamp=datetime.datetime(2026, 2, 3, 18, 51, 1, 688668) imag
e_path='/Users/maharora/Documents/Work/Other/multimodal-parser/multimodal_parser/data/images/rag_paper_bc273647-e18b-4140-a535-06365eea4a54.
png' image_format='png' image_type='diagram' image_base64='iVBORw0KGgoAAAANSUhEUgAAAxUAAADhCAIAAAAxlA/PAAEAAElEQVR4nOxdB3gqU1+Z7+xv
```

Figure 3. *Image chunk schema with vision-language description for embedding and type classification.*

Image chunks address the context gap failure mode: when figures are discarded during parsing, retrieval cannot answer visual questions. The solution pairs every extracted image with a vision-language generated description that makes visual content semantically searchable. When a document caption is available (e.g., “Figure 3: Performance comparison”), it is appended to the generated description in the `content` field, preserving figure labels for retrieval. This allows both semantic queries like “what is the system architecture” and label-based references like “Figure 3” to match against image chunks.

Images are classified by semantic type (diagram, chart, screenshot, photo, logo) to support targeted retrieval and filtering:

```python
class ImageChunk(BaseChunk):
    """Chunk containing image data."""
    chunk_type: ChunkType = Field(default=ChunkType.IMAGE)
    image_path: Optional[str] = Field(default=None, description="Extracted image path")
    image_format: Optional[str] = Field(default=None, description="png, jpg, gif, webp, etc.")
    image_type: Optional[ImageType] = Field(default="other", description="diagram, chart, etc.")
    image_base64: Optional[str] = Field(default=None, description="Base64 encoded image data")
```

Classification uses a vision-language model to distinguish diagrams, charts, screenshots, photos, and logos, enabling filtered retrieval (e.g., only architecture diagrams for system design questions). The `content` field contains retrieval-optimized descriptions emphasizing what questions the image can answer, producing semantic descriptions like “system architecture diagram showing encoder-decoder structure” rather than pixel-level enumeration.

3.3.3. *Table Chunks*

```
# Sample table chunk
if table_chunks:
    c = table_chunks[1]
    print("\n[TABLE CHUNK]")
    print(c)
```

```
[TABLE CHUNK]
chunk_id='table_36' chunk_type=<ChunkType.TABLE: 'table'> content='This table presents numerical performance scores for various Natural Lang
uage Processing (NLP) machine learning models (SotA, BART, RAG-Tok, RAG-Seq) on text generation and classification tasks. It details metric
s' sequence_number=36 source_document='rag_paper.pdf' source_page=6 parent_heading='4.1 Open-domain Question Answering' extraction_timestamp
=datetime.datetime(2026, 2, 3, 18, 51, 13, 71936) dataframe=       Model.Model Jeopardy.B-1 Jeopardy.QB-1 MSMARCO.R-L MSMARCO.B-1  \
0              SotA           –           –      49.8 *      49.9 *
1              BART        15.1        19.7        38.2        41.6
2  RAG-Tok. RAG-Seq.        17.3        22.2        40.1        41.5
3                           14.7        21.4        40.8        44.2

  FVR3.Label FVR2 Acc..Label
0       76.8          92.2 *
1       64.0            81.1
2       72.5            89.5
3       72.5            89.5   columns=None num_rows=4 num_cols=7
```

Figure 4. *Table chunk schema with DataFrame preservation and description for semantic retrieval.*

Table chunks solve the table trap: flattening a table to text destroys the relational structure that makes it valuable. A benchmark comparison table showing model performance across datasets becomes an unstructured paragraph that embeddings cannot reliably parse for

precise comparisons or metric lookups. The approach preserves the full DataFrame while adding a semantic description for retrieval:

```
class TableChunk(BaseChunk):
    """Chunk containing table data as pandas DataFrame."""
    model_config = ConfigDict(arbitrary_types_allowed=True)
    chunk_type: ChunkType = Field(default=ChunkType.TABLE)
    dataframe: pd.DataFrame = Field(description="Table data as pandas DataFrame")
    columns: Optional[List[str]] = Field(default=None, description="List of column names")
    num_rows: Optional[int] = Field(default=None, description="Number of rows")
    num_cols: Optional[int] = Field(default=None, description="Number of columns")
```

Tables require a two-stage strategy. First, semantic retrieval using generated descriptions that emphasize schema, sample data patterns, and query-answering capability (e.g., "performance metrics across five benchmark datasets"). Second, SQL execution for deterministic data access, once retrieved, the agent generates and executes queries against the DataFrame:

```
def query_table(table_index: int, sql: str) -> str:
    chunk = table_chunks[table_index]
    result_df = ps.sqldf(sql, {"df": chunk.dataframe})
    return result_df.to_string(index=False)
```

For example, a query like "Which model achieved the highest accuracy on the SQuAD benchmark?" retrieves a benchmark results table via description matching, then executes `SELECT model, accuracy FROM df WHERE dataset='SQuAD' ORDER BY accuracy DESC LIMIT 1` to extract the exact answer. This hybrid approach, semantic discovery plus deterministic execution, provides both flexibility and precision.

3.4. Indexing and Retrieval

All three chunk types flow into a unified indexing pipeline. The key insight is that text chunks, image descriptions, and table descriptions are all natural language strings that can be embedded into the same semantic space, enabling cross-modal retrieval where a text query can match against any content type. The framework uses a vector database for storing embeddings and metadata with efficient similarity search.

Indexing batch-embeds all `content` fields and stores them alongside full chunk metadata:

```
def index_file(file_path: str) -> int:
    chunks = parser.parse(file_path)
    contents = [chunk.content for chunk in chunks]
    embeddings = embed_model.embed_documents(contents)

    for chunk, embedding in zip(chunks, embeddings):
        payload = chunk.model_dump(exclude={"dataframe"})
        if chunk.chunk_type == ChunkType.TABLE:
            payload["table_data"] = chunk.dataframe.to_dict(orient="records")
        store_vector(embedding, payload)
```

For tables, the DataFrame is serialized as a dict and stored in the payload rather than embedded, since row-column data is not directly embeddable. This keeps the structured representation available for SQL execution while the description enables semantic matching.

Retrieval performs standard dense vector search. A query like "What is the proposed model architecture?" is embedded and matched against all chunk content fields, returning the top-k most similar chunks regardless of type. A diagram description like "architecture

diagram showing transformer encoder-decoder with attention mechanisms" can score highly despite being a visual element, solving the context gap by making visual content discoverable through text queries.

3.5. Agent-Based Answer Generation

Retrieval alone is insufficient for multimodal documents because different modalities require different interaction patterns. Text can be synthesized directly, but images must be displayed and tables must be queried. The framework employs a tool-augmented agent architecture [5] that combines semantic retrieval with modality-specific tools and inline rendering capabilities.

The agent receives retrieved chunks separated by type (text, image, table) and decides which operations to invoke:

- **For text chunks**: Synthesize information directly from content
- **For image chunks**: Call `display_image(image_index)` to trigger inline rendering
- **For table chunks**: Generate and execute SQL via `query_table(table_index, sql)`

3.5.1. *Inline Display Mechanism*

```
def display_image(image_index: int) -> str:
    chunk = image_chunks[image_index]
    displayed_images.append({
        "chunk_id": chunk.chunk_id,
        "path": chunk.image_path,
        "base64": chunk.image_base64,
    })
    return f"Image ready. Use placeholder [IMAGE:{len(displayed_images)-1}]
             in your answer on its own line between paragraphs."
```

Rather than returning image URLs or table data for external rendering, the agent employs a placeholder-based inline display system. When `display_image` is invoked, the tool stores the image data (base64 or file path) in a display queue and returns instructions to insert a placeholder token `[IMAGE:X]` in the answer text:

The agent is instructed to place these placeholders on separate lines, never inline within sentences, ensuring clean paragraph-image-paragraph flow in rendered output. For tables, `query_table` executes SQL and returns formatted results as text while storing the structured result in a `queried_tables` list for reference.

This tool-calling workflow enables precise operations that pure text generation cannot reliably produce. Consider a query like "What is the average F1 score across all baselines?" A text-only system must generate the answer from a flattened table representation, risking arithmetic errors. The agent retrieves the benchmark results table, generates `SELECT AVG(f1_score) FROM df WHERE model_type='baseline'`, executes it deterministically, and returns the exact result.

Similarly, visual queries like "What is the attention mechanism architecture?" retrieve an architecture diagram (via description matching) and trigger `display_image`, which inserts `[IMAGE:0]` into the answer. Downstream rendering replaces this placeholder with the actual image, presenting visual evidence directly rather than attempting to describe complex spatial relationships in generated text.

The agent workflow is:

1. Embed query and retrieve top-k chunks via semantic search
2. Separate retrieved chunks by type (text, image, table)

3. For text chunks, synthesize answer from content
4. For image chunks, invoke display tool and insert placeholder in answer
5. For table chunks, generate SQL query, execute for exact data
6. Return answer with placeholders plus image/table metadata for rendering

When retrieval returns multiple chunks of the same type, all chunks are presented to the agent in a single context, grouped by modality with numerical indices. The agent selects which chunks to act on, querying multiple tables or displaying multiple images, within a single reasoning pass. For table querying, SQL execution errors such as syntax mistakes or invalid column references are caught and returned to the agent as descriptive error messages, enabling it to revise the query and retry within the same turn.

This architecture demonstrates that multimodal RAG requires heterogeneous access patterns, text generation for prose, tool calls for structure, inline placeholders for visual elements, rather than forcing all modalities through a single generation interface.

4. Results

The framework is implemented as open-source software [6] using Gemini 2.5 Pro [7] for agent reasoning and tool calling, Gemini Embedding [8] for vector representations, Qdrant [9] for vector storage, Docling [4] for document parsing, LangGraph [10] for agent orchestration, and pandasql [11] for SQL execution against DataFrames.

4.1. Capability Demonstrations

Three demonstrations validate the framework's ability to handle queries that text-only RAG systems cannot answer. Each demonstration uses research paper PDFs as source documents and illustrates a distinct modality handling strategy.

Text Synthesis. The agent performs cross-sectional synthesis by connecting information across fragmented text chunks (Figure 5). The hybrid chunking strategy merges semantically related text blocks from the same section, enabling coherent answers that integrate information from multiple sources while avoiding over-fragmentation.

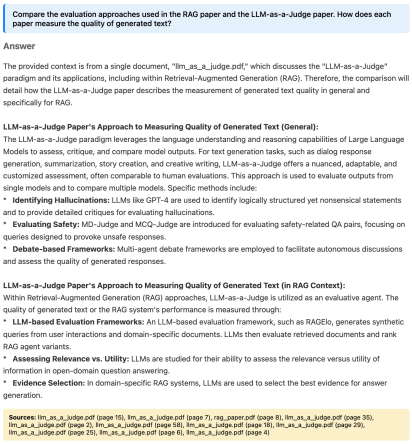


Figure 5. *Text synthesis: cross-sectional analysis connecting information across multiple chunks.*

Visual Reasoning. The agent identifies visual query intent, locates corresponding image chunks using vision-language descriptions, and displays diagrams inline (Figure 6). This addresses the context gap problem where text-only systems cannot answer queries about discarded figures, vision-language descriptions make visual content semantically searchable.

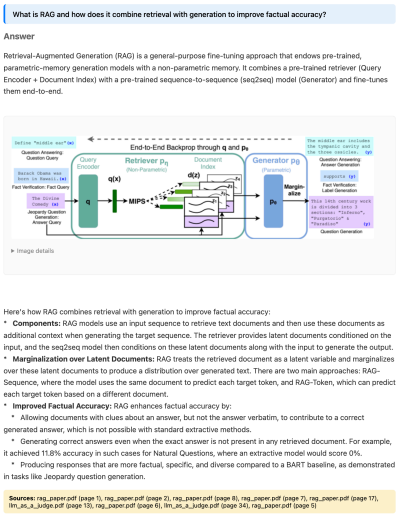


Figure 6. *Visual reasoning: diagram retrieval using vision-language descriptions and inline display.*

Structured Precision. The agent combines semantic search over table descriptions with deterministic SQL execution against DataFrames (Figure 7). This addresses the table trap problem where text-only systems hallucinate from flattened representations, the dual approach enables exact numerical results without arithmetic errors.

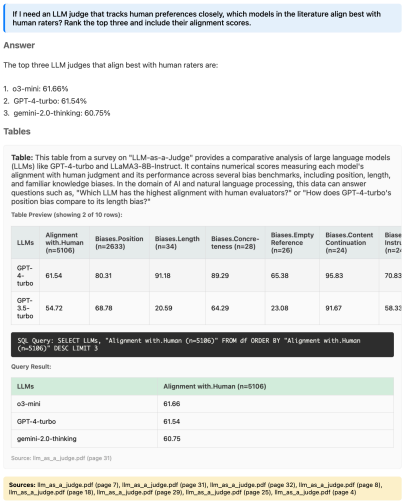


Figure 7. *Structured precision: SQL-based table querying with semantic search and deterministic execution.*

To quantify the impact of modality preservation, the multimodal pipeline was evaluated against a text-only baseline using RAGAS metrics [12]. The evaluation uses a corpus of 110 research papers [13] with 300 queries constructed across four categories: text-only (28%), table (28%), image (24%), and cross-modal (20%) queries requiring synthesis across modalities. The text-only baseline uses identical chunking and retrieval parameters but converts tables to inline Markdown text and discards images during ingestion. Results are averaged over three independent runs; 95% confidence intervals are reported.

Table 1. *Multimodal (MM) vs. text-only (TO) pipeline comparison by query type. CR = Context Recall, F = Faithfulness, AC = Answer Correctness. Overall values show mean ± 95% CI over three runs.*

Query Type	CR (MM)	CR (TO)	F (MM)	F (TO)	AC (MM)	AC (TO)

Text	91.8%	91.0%	95.2%	94.5%	79.6%	78.8%
Table	89.2%	82.3%	94.8%	90.3%	76.5%	69.2%
Image	83.5%	64.7%	92.5%	84.8%	70.3%	60.5%
Cross-modal	77.4%	74.2%	90.7%	89.5%	66.8%	64.7%
Overall	**86.2 ± 2.3%**	**78.9 ± 1.3%**	**93.6 ± 1.4%**	**90.0 ± 0.9%**	**74.0 ± 2.4%**	**68.9 ± 1.7%**

The per-modality breakdown reveals that gains concentrate on image queries, where the baseline discards the primary evidence source, context recall improves by 18.8 percentage points. Table queries show a moderate gap (6.9 pp in context recall) since the baseline retains table content as inline text but lacks deterministic SQL access for precise lookups. Text queries show minimal difference (0.8 pp), confirming that the multimodal pipeline preserves text handling quality.

These results validate that multimodal RAG requires both modality-specific representations and heterogeneous access patterns. The bottleneck has shifted from LLM reasoning to data quality and modality preservation.

5. Discussion

5.1. Runtime Feasibility

The SQL execution pathway requires materializing each retrieved table as an in-memory DataFrame at query time. For individual document tables, typically tens to hundreds of rows, this overhead is negligible, and the execution engine (pandasql over SQLite) operates in-process without external infrastructure. At larger corpus scales, tables and images can be stored in external storage (e.g., S3 or cloud drives) as CSV and image files respectively, rather than serialized inline in vector payloads. The vector store retains only metadata paths, and content is loaded on demand when the agent invokes a tool, with caching for frequently accessed tables to avoid repeated retrieval. The primary computational cost lies in ingestion rather than retrieval: vision-language captioning and table description generation require LLM inference per element, making ingestion time proportional to the number of non-text elements in the corpus.

5.2. Limitations

The framework's retrieval quality for images and tables depends on the fidelity of their generated descriptions. These descriptions are produced once during ingestion and stored as static text, if the vision-language model omits details present in the original image or table, those details become unretrievable through semantic search. Re-ingestion with an improved captioning model is required to recover missed information. Additionally, the agent's ability to answer table queries depends on correct SQL generation; while the framework supports error-driven retry, persistent failures on tables with ambiguous or non-standard schemas cause the agent to fall back to the natural language description, sacrificing deterministic precision. Complex table structures such as merged cells or nested headers remain challenging for the underlying structure recognition model.

6. Conclusion and Future Work

This paper presents a multimodal document parsing framework for RAG systems that addresses the fundamental limitations of text-only approaches. By preserving document structure through specialized chunking strategies for text, images, and tables, the system

enables retrieval of visual and structured information that traditional pipelines discard. The results demonstrate that the bottleneck in RAG performance has shifted from LLM reasoning capabilities to data quality and modality preservation, modality-specific chunking enables retrieval relevance that pure text embeddings cannot achieve, while tool-calling architectures prove essential for handling heterogeneous access patterns across text generation, image display, and SQL execution. Future work should develop comprehensive multimodal RAG benchmarks, extend the framework to video and equations, and explore domain-specific adaptations for scientific literature and technical documentation. The open-source implementation [6] facilitates community adoption and experimentation.

References

[1] R. Zhao *et al.*, "Retrieving Multimodal Information for Augmented Generation: A Survey," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 4736–4756. doi: 10.48550/arXiv.2303.10868.

[2] LangChain AI, "Multi-Vector Retriever." [Online]. Available: https://python.langchain.com/docs/how_to/multi_vector/

[3] LlamaIndex, "LlamaIndex: Data Framework for LLM Applications." [Online]. Available: https://docs.llamaindex.ai/

[4] N. Livathinos *et al.*, "Docling: An Efficient Open-Source Toolkit for AI-driven Document Conversion," *arXiv preprint arXiv:2501.17887*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17887

[5] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," *arXiv preprint arXiv:2210.03629*, 2022, doi: 10.48550/arXiv.2210.03629.

[6] M. Arora, "Multimodal Parser: Document Parsing for RAG." [Online]. Available: https://github.com/mahimaarora/multimodal-parser

[7] G. Comanici, E. Bieber, M. Schaekermann, and others, "Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities," *arXiv preprint arXiv:2507.06261*, 2025, doi: 10.48550/arXiv.2507.06261.

[8] J. Lee *et al.*, "Gemini Embedding: Generalizable Embeddings from Gemini," *arXiv preprint arXiv:2503.07891*, 2025, doi: 10.48550/arXiv.2503.07891.

[9] Qdrant Team, "Qdrant: Vector Database for the Next Generation of AI Applications." [Online]. Available: https://qdrant.tech/

[10] LangChain AI, "LangGraph: Build resilient language agents as graphs." [Online]. Available: https://github.com/langchain-ai/langgraph

[11] pandasql Contributors, "pandasql: Query pandas DataFrames using SQL syntax." [Online]. Available: https://github.com/yhat/pandasql

[12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation," *arXiv preprint arXiv:2309.15217*, 2023, doi: 10.48550/arXiv.2309.15217.

[13] M. Arora, "Research Papers Corpus for Multimodal RAG Evaluation." [Online]. Available: https://huggingface.co/datasets/mahimaarora025/research_papers